

Lab 0 Extra

准备工作

请在**自动初始化分支**后，在开发机依次执行以下命令：

```
$ cd ~/学号
$ git fetch
$ git checkout lab0-extra
```

题目背景

日志是排查问题的重要依据，系统管理员需要定期分析 Web 服务器的日志。

文件说明

服务器每天会产生两类日志：

- 访问日志（`access.log`）：记录 HTTP 访问请求，每行格式如下：

```
IP地址 - - [时间 时区] 请求方法 请求路径 协议 状态码 响应大小
```

例如：

```
172.16.0.3 - - [2025-05-21:00:14:41 +0800] DELETE /images/logo.png HTTP/2.0 404
768
```

- 错误日志（`error.log`）：记录服务器错误信息，每行格式如下：

```
日期 时间 [级别]：消息
```

例如：

```
2025-07-06 02:02:05 [INFO]: Client disconnected
```

格式说明中的不同字段之间以空格分隔，除 `消息` 字段外，字段内部均不含空格。

服务器的日志根目录为 `logs`，日志被保存在以对应日期命名的子目录中，结构如下：

```
logs/
├── YYYY-MM-DD/
│   ├── access.log
│   └── error.log
└── ...
```

保证日志根目录不为空，且每个子目录中均存在两类日志文件，但不保证日志内容不为空。

题目要求

脚本： `analyze.sh`

用法： `bash analyze.sh 日期`

功能：分析指定日期的日志，日期格式为 `YYYY-MM-DD`

1. 如果 `日期` 未给出，则打印 `analyze.sh: No date provided` 到标准输出，并**立即退出**。

如果 `日期` 给出，则按如下要求分析对应日期（保证存在）的日志。

2. 新建报告根目录 `reports`，与日志根目录同级，报告均保存在以对应日期命名的子目录中。

新建**所有目录**的权限为 `rw-rwxr-x`，**所有普通文件**的权限为 `rw-rw-r--`。

3. 在指定日期的错误日志中检查是否存在含有 `ERROR` 字样的记录，如果有则将这类记录筛选出来，按原顺序保存到相应报告子目录下的 `error.log`，否则**不要创建此文件**。

4. 在指定日期的访问日志中将 `/127.0.0.1` 字样全部替换为 `/localhost`，其它内容保持不变，全文保存到相应报告子目录下的 `access.log`，**不要更改原文件**。

5. 在指定日期的访问日志中统计每个 `IP地址` 的总请求次数，并将结果按访问次数从高到低排序，保存到相应报告子目录下的 `summary.txt`，访问次数相同时的排序**不做要求**，每行格式如下：

```
IP地址 总访问次数
```

提交时，**只应提交脚本文件本身**，不应提交你测试时产生的临时文件。

运行后，脚本**不应执行多余操作**，标准输出、标准错误和提交目录中不能有多余内容，可能的一种目录结构如下：

```
logs/
├── YYYY-MM-DD/
│   ├── access.log
│   └── error.log
└── ...
reports/
├── YYYY-MM-DD/
│   ├── access.log
│   ├── error.log
│   └── summary.txt
analyze.sh
```

提交评测 & 评测标准

请在开发机中执行下列命令后，在课程网站上提交评测。

```
$ cd ~/学号/
$ git add analyze.sh
$ git commit -m "message" # 请将 message 改为有意义的信息
$ git push
```

测试点和分数说明如下：

共 10 个测试点，每个测试点 10 分。测试用例与给出的样例不同，每个测试点的主要评测内容如下所示。

测试点序号	评测内容
1, 2	错误处理
3	权限设置
4, 5	查找筛选
6, 7	全局替换
8, 9, 10	排序计数

提示

- 1. 脚本参数用法详见指导书第 34 页
- 2. 流程控制语句用法详见指导书第 35 页
- 3. 权限设定命令用法详见指导书第 31 页
- 4. `sed` 会将模式字符串当作正则表达式解析，如果需要将特殊符号按原字符解析则应使用反斜杠 `\` 转义。

替换命令 `s` 允许使用其它字符作为分隔符，分隔符如果出现在模式或替换字符串中也应该使用反斜杠 `\` 转义。

例如： `'s/\./\./g'` 和 `'s|/./|\\.|g'` 等价，都可以将所有 `/.` 替换为 `\.`

- 5. `sort`: 连接所有文件，然后排序，并将结果写到标准输出

用法: `sort [选项]... [文件]...`

选项: `-r` 逆序输出排序结果、`-n` 按数值大小而非字典序进行排序

- 6. `uniq`: 从输入文件或标准输入中过滤内容相同的相邻的行，并写到输出文件或标准输出

用法: `uniq [选项]... [输入文件 [输出文件]]`

选项: `-c` 在每行之前加上该行的重复次数作为前缀

参考代码框架如下（实现方法不止一种）：

```
#!/bin/bash

LOG_DIR="logs"
REPORT_DIR="reports"

# 如果缺少参数
if [ ]; then
    # 打印消息，立即退出
fi

# 获取日期参数
DATE=

# 构建文件路径
```

```
error_src=
access_src=
error_dst=
access_dst=
summary_dst=

# 查找包含 ERROR 的行
errors=$(grep TODO "$error_src")
if [ ]; then
    # 仅当结果非空时保存
fi

# 思考如何处理特殊字符
sed TODO "$access_src"

# 思考综合使用提示命令
awk TODO "$access_src" | TODO | TODO | TODO | TODO
```